

Scheda — FI2 LAB02: ADT Frazione, prima parte

Materiale: LAB-02-Frazione-PrimaParte.pdf (sl. 1-25), Lab02-Frazione-PrimaParte-Startkit.zip · **Modalità:** guidata **Teoria che entra qui:** 04b □ — classi come ADT sl. 40-44, costruttori sl. 77-82, Frazione *all together* sl. 85; 06 □ — toString e @Override sl. 24-26; S01 □ (Eclipse: la procedura serve qui, sl. 15-25 del LAB).

Setup

- scompatta lo zip, **rinomina la cartella**, *File → Import → Existing Projects into Workspace*, poi **Refactor → Rename** del progetto con l'aggiornamento dei riferimenti spuntato — «All'esame è espressamente richiesto!» [fonte: LAB02 sl. 15-22];
- niente package: tutto in src/ (default package), come nello startkit;
- le X rosse sono normali: mancano Frazione e il corpo di MyMath.mcm [sl. 23];
- i «test» qui **non sono JUnit**: sono assert dentro il main di FrazioneTest → *Run As → Run Configurations → Arguments → VM arguments = -ea* [sl. 12, 24-25].

Ordine di lavoro

#	Cosa	Cosa rappresenta	Test che la coprono	Costrutto nuovo → slide
1	MyMath.mcm(int, int)	comune multiplo; mcd è già dato	nessuno (serve a compilare)	metodo static di libreria — LAB01
2	Frazione: campi + due costruttori	valore immutabile, segno sul solo numeratore	blocchi «costruzione» e «valori negativi»	private, costruttore primario/ausiliario, this(n, 1) — 04b sl. 79-82, LAB02 sl. 5-6
3	getNum(), getDen()	accessor; niente set*	gli stessi	accessor — LAB02 sl. 7
4	equals(Frazione f)	equivalenza , non identità: $n \cdot q = m \cdot p$	«funzionamento equals»	LAB02 sl. 8
5	minTerm()	nuova frazione ridotta, l'originale non cambia	«riduzioneMinimiTermini»	Tesi di MyMath.mcd — LAB02 sl. 9

#	Cosa	Cosa rappresenta	Test che la coprono	Costrutto nuovo → slide
6	toString()	forma Num/Den	nessun assert: controlla a occhio l'output di MainFrazione	@Override — 06 sl. 24-26

Firme dall'UML (sl. 11): `Frazione(num: int, den: int)`, `Frazione(num: int)`, `getNum(): int`, `getDen(): int`, `equals(f: Frazione): boolean`, `minTerm(): Frazione`, `toString(): String`.

Casi limite da pensare

- `new Frazione(2, -3)` deve dare $-2/3$, `new Frazione(-5, -7)` deve dare $5/7$: dove decidi il segno, e con quale condizione copri **tutti e quattro** i casi di segno?
- `mcd` «prevede numeri NATURALI» [sl. 9]: cosa gli passi quando riduci $-2/3$?
- E se il numeratore è 0? Segui a mano `mcd(0, 5)` nel codice dato: cosa succede alla riga `a % b`?
- `equals` fra $3/12$ e $1/4$ deve dare `true` senza ridurre nessuna delle due: perché la formula di sl. 8 non ne ha bisogno?
- Immutabile: dopo `minTerm()`, `frazione1` vale ancora $3/12$?
- ⚠ La classe è finita quando passano **tutti** i blocchi di `FrazioneTest`, non il primo (profilo/errori.md, pattern 2).

Test rosso?

Non compila (firma, nome, return mancante) · `AssertionError = failure`: la riga dello stack trace indica l'assert che non torna · altra eccezione (es. `ArithmeticException: / by zero`) = error → prontuario §1, oppure scrivilo in chat. ⚠ **Silenzio = verde solo con -ea attivo**: senza, gli assert non vengono eseguiti e il programma termina muto anche con `Frazione` sbagliata. Collauda metodo per metodo commentando i blocchi non pertinenti [sl. 23].

Dopo i test verdi

- copia il progetto in `corsi/FI2/esame_FI2/svolti/LAB02_Frazione/`;
- apri la soluzione del docente e confronta: come normalizza il segno, come tratta il numeratore 0, cosa stampa `toString` quando il denominatore è 1;
- /chiudi aggiorna il prontuario (§2: import, rinomina, -ea).